Lecture 1: The Waterfall Model

Software Programming vs Development vs Engineering

- Programming = creating steps a computer can run to solve a problem (algorithm + program)
- Development = break the problem into simpler sub-problems, then program
- Engineering = make sure the software meets the requirements, with emphasis on quality (analysis, development, programming, testing)

Process

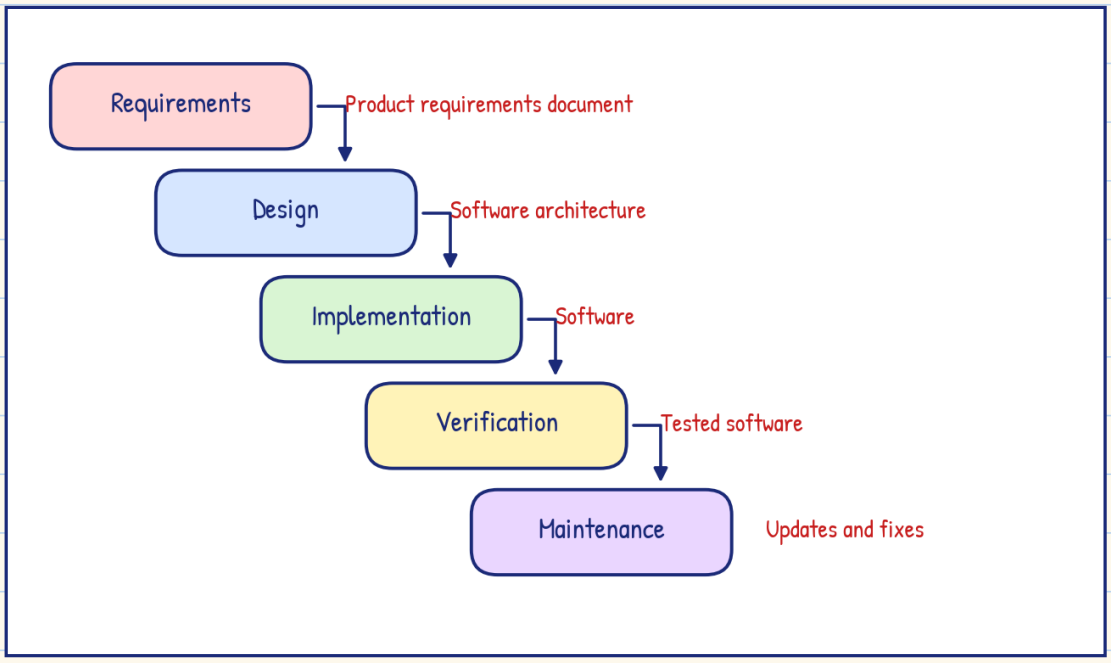
- A process is a series of steps (activities, constraints, resources) that produces an intended output
- Uses tools and techniques
- Why use a process:
  - Gives consistency and structure
  - Helps us understand, control, examine and improve activities
  - Lets us capture experience and pass it on

Process Characteristics

- Lists all major activities and uses resources under constraints (e.g. schedule)
- Produces intermediate and final products
- Each activity has entry and exit criteria
- Activities are in sequence, so timing is clear
- Has guiding principles and goals for each activity

Waterfall Model

- One of the first process models: phases follow one after another, each ending with milestones and deliverables
- Works for well-understood problems with few or no requirement changes
- Simple and easy to explain to customers
- Steps (phases) in order:
  - 1. Requirements -> product requirements document
  - 2. Design -> software architecture
  - 3. Implementation -> software
  - 4. Verification
  - 5. Maintenance



Stakeholders and Roles

- Stakeholders are all the people involved in or affected by the project (customers, users, managers, developers, testers)
- Each stakeholder and engineering role takes part in different waterfall phases

Lecture 2: Modelling the Process and Life Cycle

Reasons for Modelling a Process

- To form a common understanding
- To find inconsistencies, redundancies, omissions
- To evaluate activities for reaching process goals
- To tailor a general process to a situation

Software Life Cycle (steps)

- 1. Requirements analysis and definition
- 2. System (architecture) design
- 3. Program (detailed) design
- 4. Coding / implementation
- 5. Testing: unit, integration, system
- 6. Delivery (deployment)
- 7. Maintenance

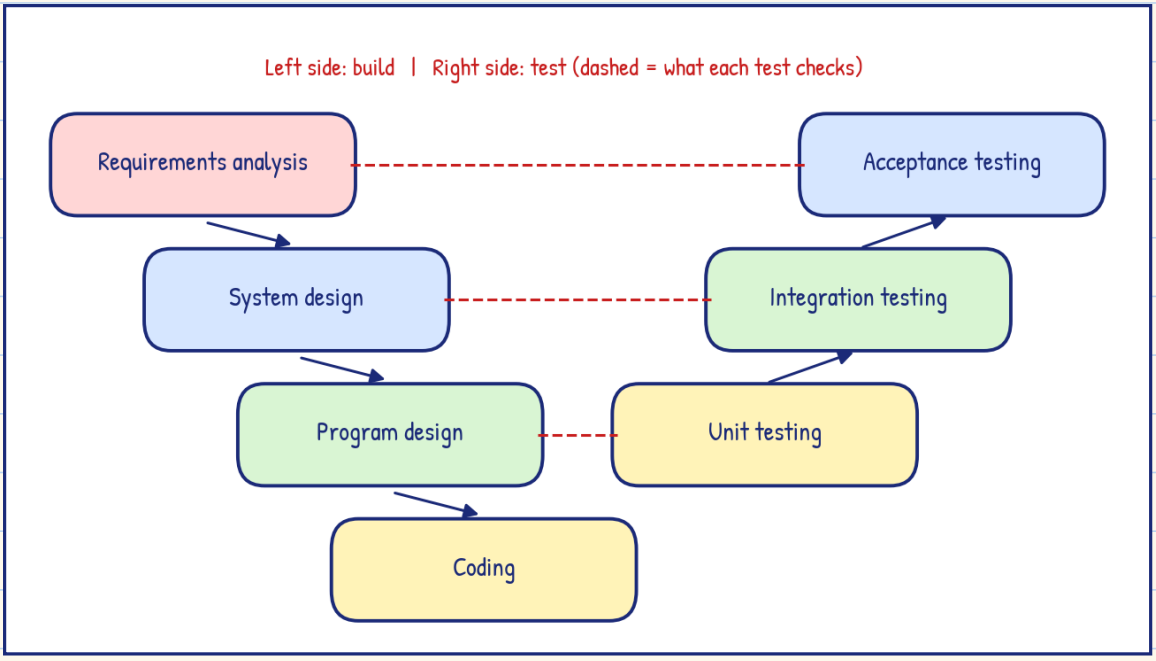
Waterfall Model - Drawbacks

- No guidance for handling changes (assumes requirements are frozen)

- Treats software like manufacturing, not a creative process
- No iteration, and long wait for the final product

Prototype and V Model

- Prototype = a partially developed product; helps developers compare designs and users see the interface
- V model = waterfall variation where each design stage is matched with a test stage
  - Unit testing verifies program design
  - Integration testing verifies system design
  - Acceptance testing validates the requirements



Prototyping Model

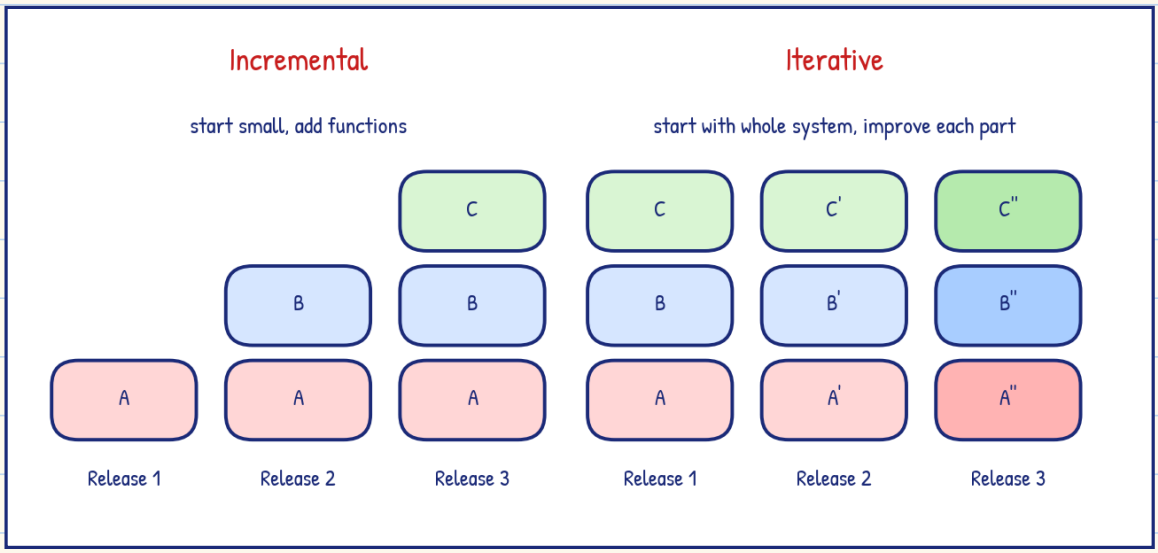
- Allows repeated investigation of the requirements or design
- Reduces risk and uncertainty in development

Other Models

- Operational specification = requirements are executed early to see their implications
- Transformational = a formal specification is changed step by step into the system (change data, select algorithms, optimize, compile)

Phased Development

- System is delivered in pieces; release n is in use while release n+1 is being built
- Incremental = start with a small working subsystem and add functions each release
- Iterative = start with the whole system and improve each subsystem each release

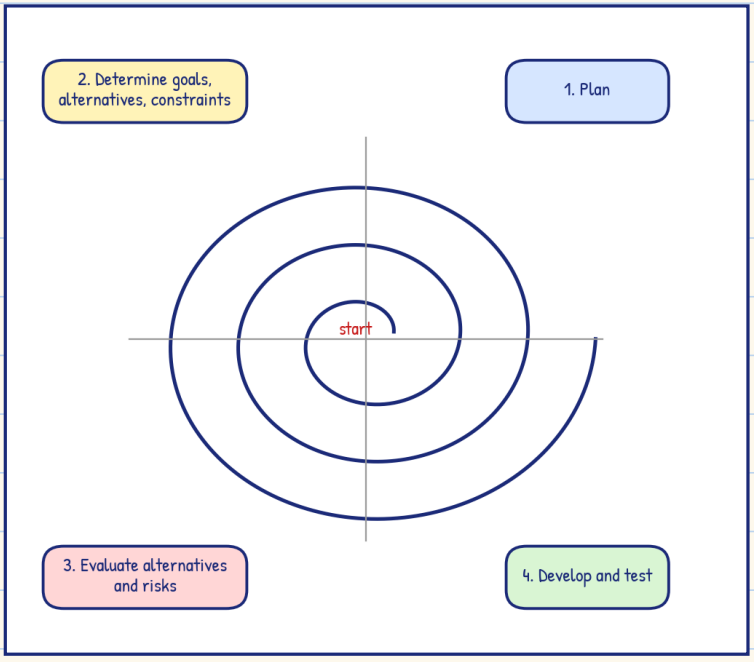


Features of Phased Development

- Training can begin early, even if some functions are missing
- Markets can be created early for new functions
- Frequent releases let developers fix problems globally and quickly
- Team can focus on different areas of expertise in different releases

Spiral Model (Boehm, 1988)

- Development combined with risk management; each loop of the spiral has 4 steps
- 1. Plan
- 2. Determine goals, alternatives and constraints
- 3. Evaluate alternatives and risks
- 4. Develop and test



Agile Methods

- Agile = flexibility in producing software quickly and capably
- Agile manifesto values:
  - Individuals and interactions over process and tools
  - Working software over comprehensive documentation
  - Customer collaboration over contract negotiation
  - Responding to change over following a plan
- Examples: Extreme Programming (XP), Crystal, Scrum (30-day iterations, daily scrum), ASD
- XP values: Communication, Simplicity, Courage, Feedback

Extreme Programming (XP)

- Based on four values: Communication, Simplicity, Courage, Feedback
- 12 facets: planning game, small releases, metaphor, simple design, tests first, refactoring, pair programming, collective ownership, continuous integration, sustainable pace (40 hours/week), on-site customer, coding standards
- Weakness: the practices depend on each other, so changing one makes the method vulnerable; passing tests may not be what the customer pays for; refactoring can degrade the architecture

Process Modelling: Static vs Dynamic

- Static model = shows the structure of the process (a snapshot), e.g. Lai notation
- Dynamic model = enacts the process to see what happens over time, e.g. System Dynamics
- Lai notation views a process as 7 types: Activity, Sequence, Process model, Resource, Control, Policy, Organization

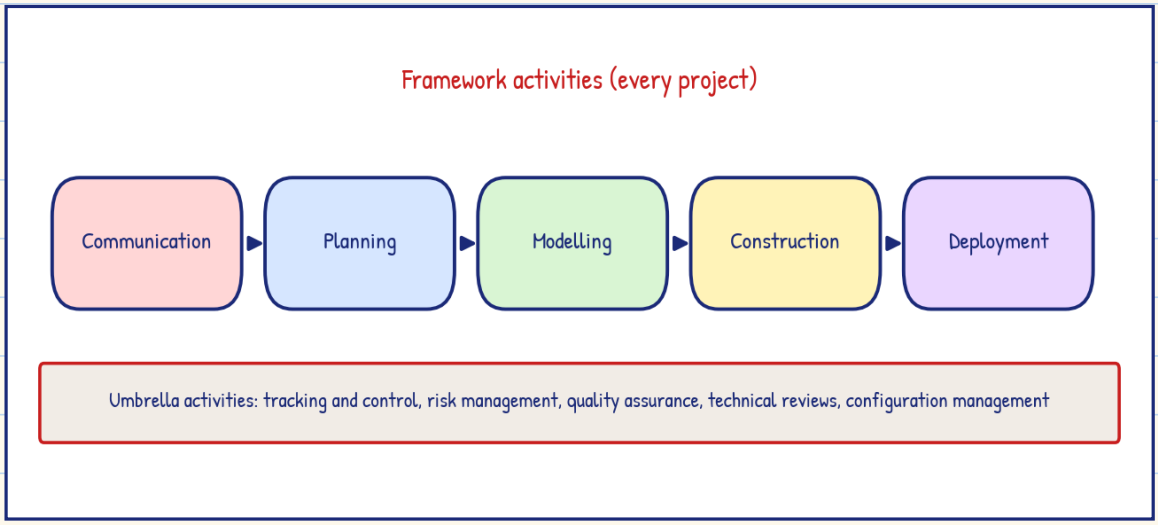
System Dynamics Model

- Introduced by Forrester in the 1950s; Abdel-Hamid and Madnick applied it to software development
- Pictorial model of factors affecting productivity; arrows show how a change in one factor changes another
- Example: how the development process affects productivity

Process Framework

- Framework activities apply to every project; umbrella activities run through the whole project
- 5 framework activities (steps):
  - 1. Communication – work with customer, gather requirements
  - 2. Planning – make the project plan: tasks, risks, resources, schedule
  - 3. Modelling – sketch requirements and design
  - 4. Construction – code and test

    - 5. Deployment – deliver to customer and get feedback
- Umbrella activities: project tracking and control, risk management, quality assurance, technical reviews, configuration management

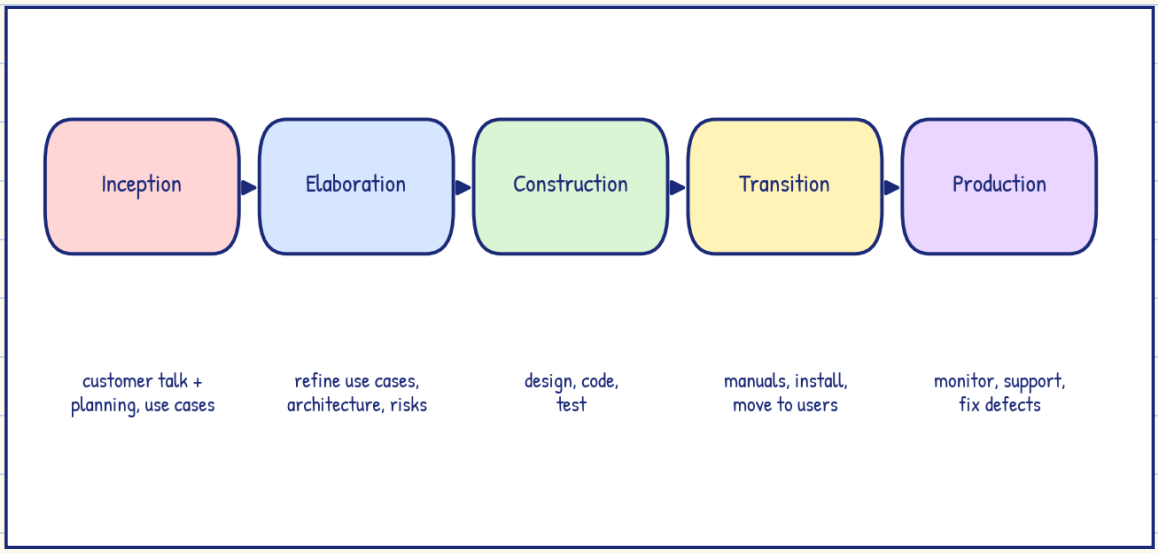


Example: Piccadilly Television

- Advertising program that must be easy to maintain and change; most uncertainty is in regulations and business constraints
- Waterfall is too rigid, so the Spiral model with prototyping of the user interface is chosen
- Spiral lets us revisit assumptions, analyse risks and prototype characteristics

Unified Process

- Use-case driven, architecture-centric, iterative and incremental process using UML
- Phases (steps):
    - 1. Inception – customer communication and planning, preliminary use cases, rough architecture
    - 2. Elaboration – refine use cases, understand the problem, set architecture, plan, find key risks
    - 3. Construction – design, program and test; make each use case work
    - 4. Transition – move to users: manuals, troubleshooting guides, installation
    - 5. Production – monitor use, support, handle defect reports and change requests



Lecture 3: Requirements Engineering - I

Requirement

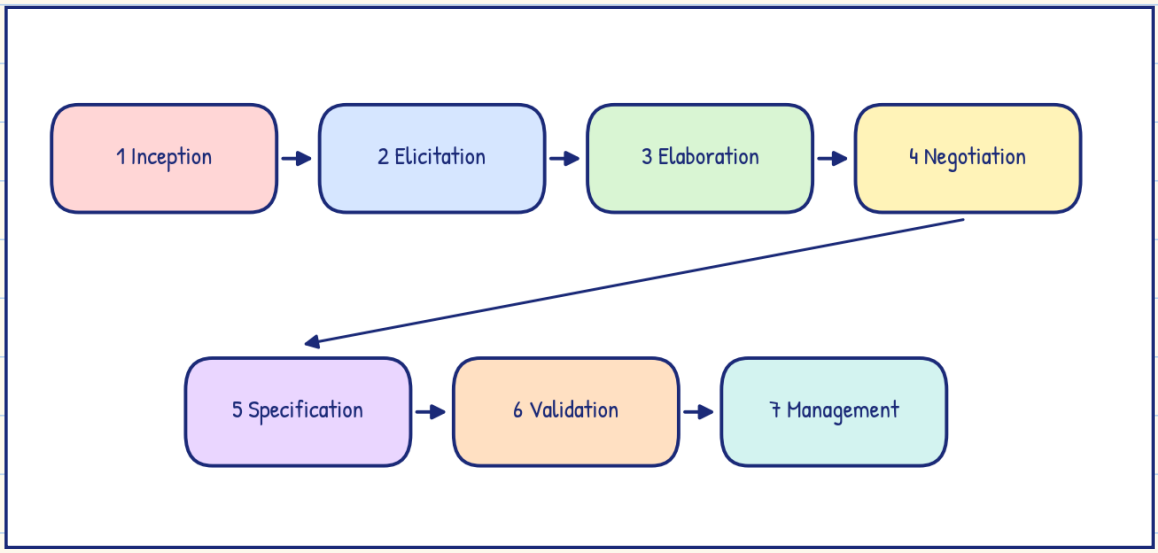
- A requirement is an expression of desired behavior: what the system must do, not how
- Deals with objects, their states, and functions that change states

Requirements Engineering (RE)

- RE = the tasks and techniques that lead to an understanding of requirements
- Final output: Software Requirements Specification (SRS), made by the requirements analyst

RE Tasks (step by step)

- 1. Inception - basic understanding of problem and solution
- 2. Elicitation - draw requirements out of stakeholders
- 3. Elaboration - build an analysis model (information, function, behavior)
- 4. Negotiation - agree on a realistic system
- 5. Specification - describe the requirements
- 6. Validation - review for errors, ambiguities, omissions, conflicts
- 7. Requirements management - handle changing requirements



Inception

- Ask questions to establish a basic understanding of the problem, who wants a solution, the nature of the solution, and how well customer and developer communicate

Elicitation

- Stakeholders: clients (pay), customers (buy), users, domain experts, market researchers, lawyers/auditors, software engineers
- Means: interviews, reading documents, observing current system, apprenticing with users, group interviews, brainstorming
- Why it is difficult: customers are unsure of needs; unclear scope; requirements change (volatility)

Elaboration

- Develop a refined technical model of functions, features and constraints from inception and elicitation
- Creates an analysis model of data, function and behavior, driven by user scenarios showing how the end-user interacts with the system

Prioritizing Requirements

- 1. Essential - must be met
- 2. Desirable - highly wanted but not necessary
- 3. Optional - could be dropped

Example: Credit Card Billing System

- Essential: list current charges, sum them and request payment by a date
- Desirable: separate charges by purchase type
- Optional: print credits in black and debits in red

Functional vs Non-functional

- Functional requirement = required behavior: activities, reactions to inputs, data formats
- Non-functional (quality) requirement = a quality the software must have
  - Examples: performance, usability, security, reliability, availability, maintainability, accuracy, cost/time to delivery

Negotiation and Validation

- Negotiation: categorize, find relations, review, prioritize, discuss cost and timeline - no winner, no loser
- Validation: formal technical review for errors, missing information, inconsistencies, unrealistic requirements

Specification

- Can be a written document, graphical models, a formal mathematical model, usage scenarios, a prototype, or a combination
- Large systems: written document and graphical models; small systems: usage scenarios

Characteristics of Good Requirements

- Correct, Consistent, Clear/Unambiguous, Complete
- Feasible, Relevant, Testable, Traceable

Lecture 4: Requirements Specification (SRS)

Requirements Specification

- Restates requirements as how the system shall behave; written for designers, testers and project managers by the analyst
- No information may be lost or changed

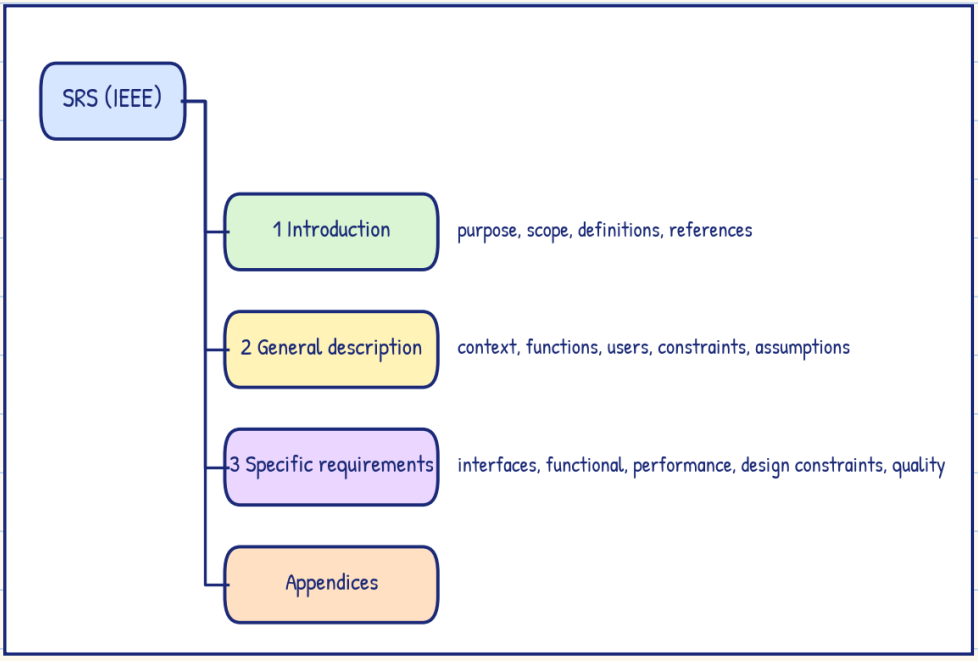
Steps to document

- 1. Describe all inputs and outputs: sources, destinations, value ranges, data formats, protocols, window formats, timing constraints
- 2. Restate the required functionality in terms of those inputs and outputs

IEEE SRS Structure

- 1. Introduction: purpose, scope, definitions, references, outline
- 2. General description: context, product functions, user characteristics, constraints, assumptions and dependencies
- 3. Specific requirements: external interfaces (user, hardware, software, communication), functional, performance, design constraints, quality, other

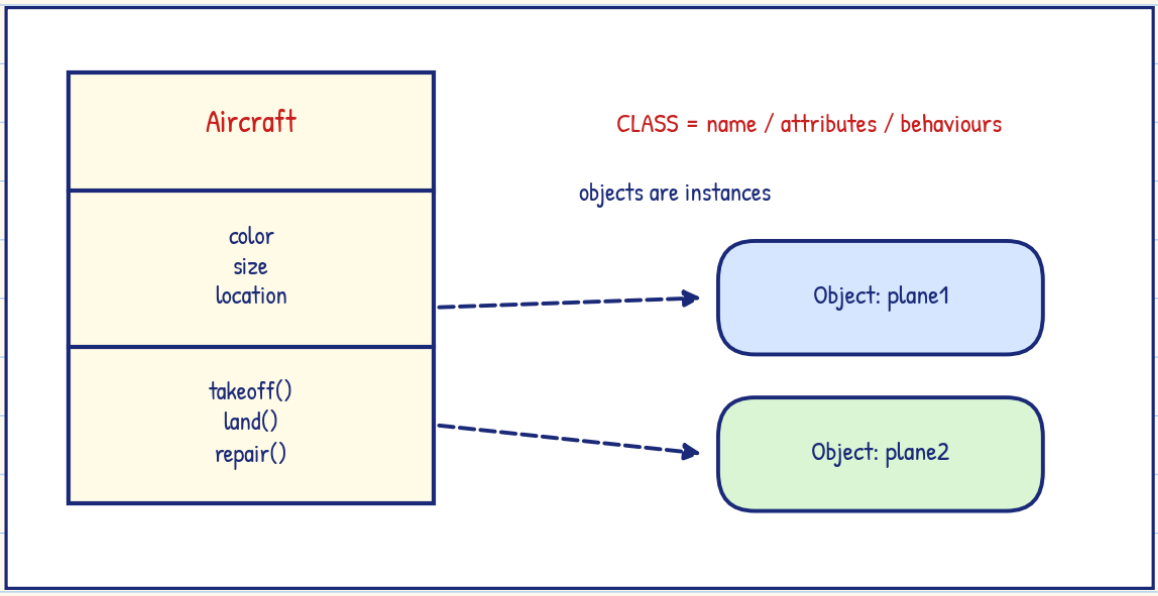
- 4. Appendices
- SRS must be clear, unambiguous and complete



Lecture 5: OO Analysis and Design Through UML

Object, Class, Method

- Object = runtime entity: data plus operations; has attributes and behaviors
- Class = group of objects with common attributes and behaviors; an object is an instance of a class
- Method = a specific implementation of an operation for a class
- OO design splits a system into objects that encapsulate data and function; reuse through inheritance



OO Development (steps)

- 1. OO requirements - in the user's language
- 2. System design - find classes, attributes, behaviors, relationships
- 3. Program design - add computation details and non-functional needs
- 4. Coding - translate models into an OO language
- 5. Testing

Model

- Model = a simplification of reality; the blueprint of a system (structural or behavioral)
- Models help us visualize, specify, guide construction and document decisions

Aims of Modelling

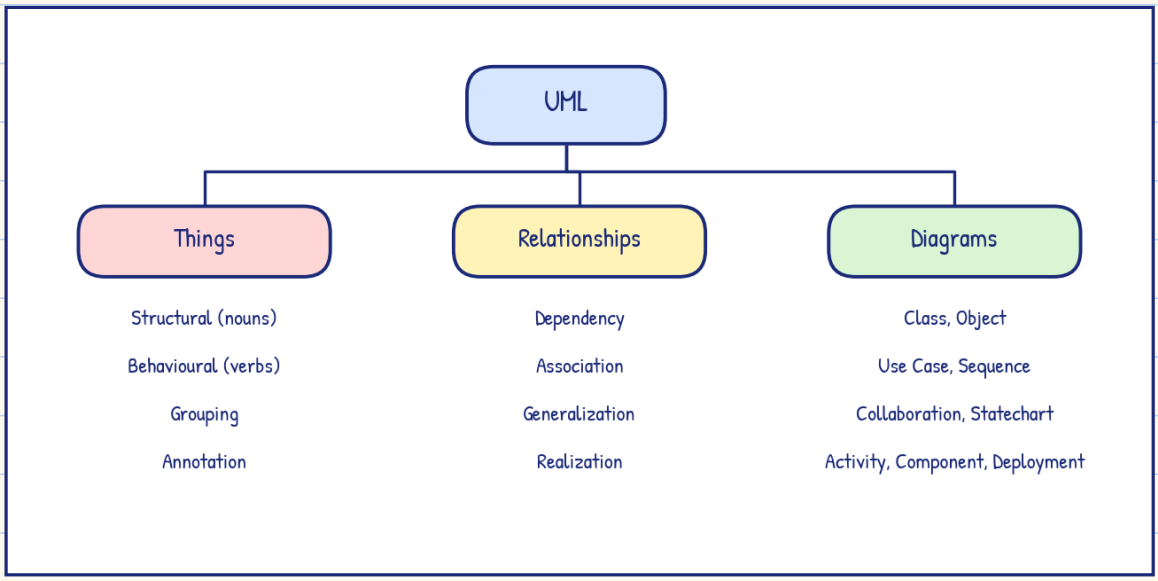
- Visualize a system as it is or as we want it to be
- Specify the structure or behavior of the system
- Give a template that guides construction
- Document the decisions we have made

Two Ways to Model

- Algorithmic perspective: building blocks are procedures or functions; gets hard to maintain as requirements change
- Object-oriented perspective: building blocks are objects or classes; every object has identity, state and behavior

UML

- UML = family of graphical notations for describing and designing software, especially OO systems; process independent
- Used to Visualize, Specify, Construct, Document
- Created by Booch, Jacobson, Rumbaugh; controlled by OMG
- 3 building blocks: Things, Relationships, Diagrams



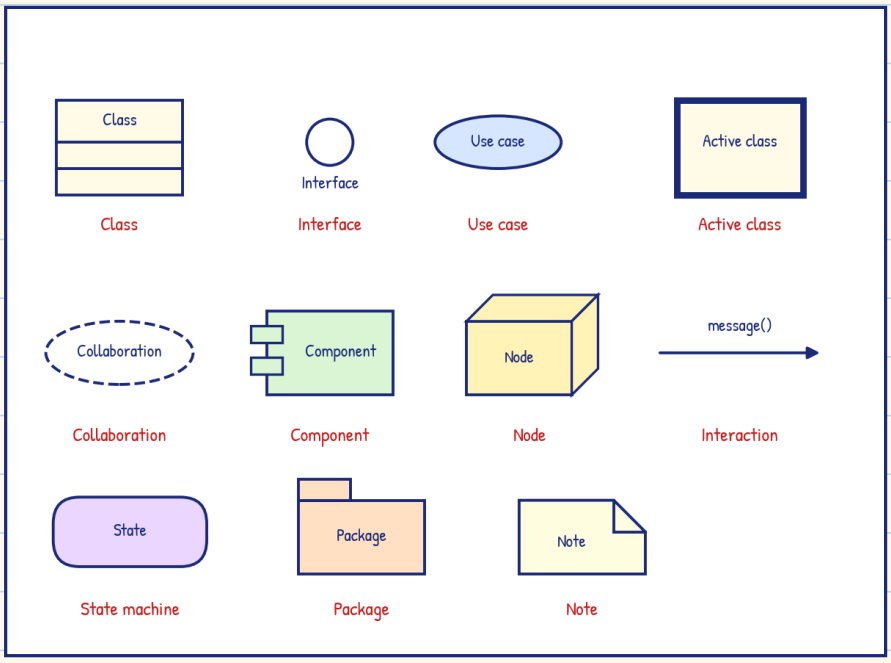
Lecture 6: UML Diagrams

Basic Building Blocks of UML

- UML has three kinds of building blocks:
  - Things / elements - abstractions that are the first-class citizens of a model
  - Relationships - tie things together
  - Diagrams - group interesting collections of things

Things in the UML

- There are four kinds of things:
  - Structural things - the nouns of UML models; mostly static parts; represent conceptual or physical elements
  - Behavioral things - the dynamic (verb) parts of UML models
  - Grouping things - the organizational parts of UML models
  - Annotational things - the explanatory parts of UML models



Structural Things (7 kinds)

- Class - a description of a set of objects that share the same attributes, relationships and semantics; a class implements one or more interfaces; drawn as a rectangle
- Interface - a collection of operations that specify a service of a class or component; describes the externally visible behavior of that element; rarely stands alone, it is attached to a class or component; drawn as a circle with its name
- Use case - a description of a set of sequences of actions that a system performs to yield an observable result to a particular user; drawn as a solid ellipse with its name
- Active class - a class that can initiate control activity; its objects represent elements whose behavior is concurrent with other elements and own one or more processes or threads; drawn like a class with heavy lines
- Collaboration - defines an interaction; a society of roles and other elements that work together to provide behavior bigger than the sum of its elements; has structural and behavioral dimensions; drawn as an ellipse with dashed lines
- Component - represents a physical thing; a physical and replaceable part of a system; drawn as a rectangle with tabs
- Node - a physical element that exists at run time; represents a computational resource having at least one memory; drawn as a cube

Class

- Name - every class must have a name that distinguishes it from other classes
- Attribute - a named property of a class that describes a range of values that instances of the

property may hold; a class may have any number of attributes, or none at all

- Operation - the implementation of a service that can be requested from any object of the class to affect behavior
- Attribute notation: a property written as a line of text inside the class box
- Full form of an attribute: visibility name: type [multiplicity] = default {property-string}
  - Example: - name: String [1] = "Untitled" {readOnly}
  - Only the name is necessary

Behavioral Things

- The verbs of a model; they represent behavior over time and space. Two types:
- Interaction - a behavior made of a set of messages exchanged among a set of objects within a particular context to get a result; drawn as a directed line with the name of its operation
- State machine - a behavior that specifies the sequence of states an object or an interaction goes through during its lifetime in response to events; can specify the behavior of a class or collaboration; drawn as a rounded rectangle with its name and sub-states
- A state machine involves:
  - States
  - Transitions - the flow from state to state
  - Events - things that trigger a transition
  - Activities - the response to events

Grouping Things

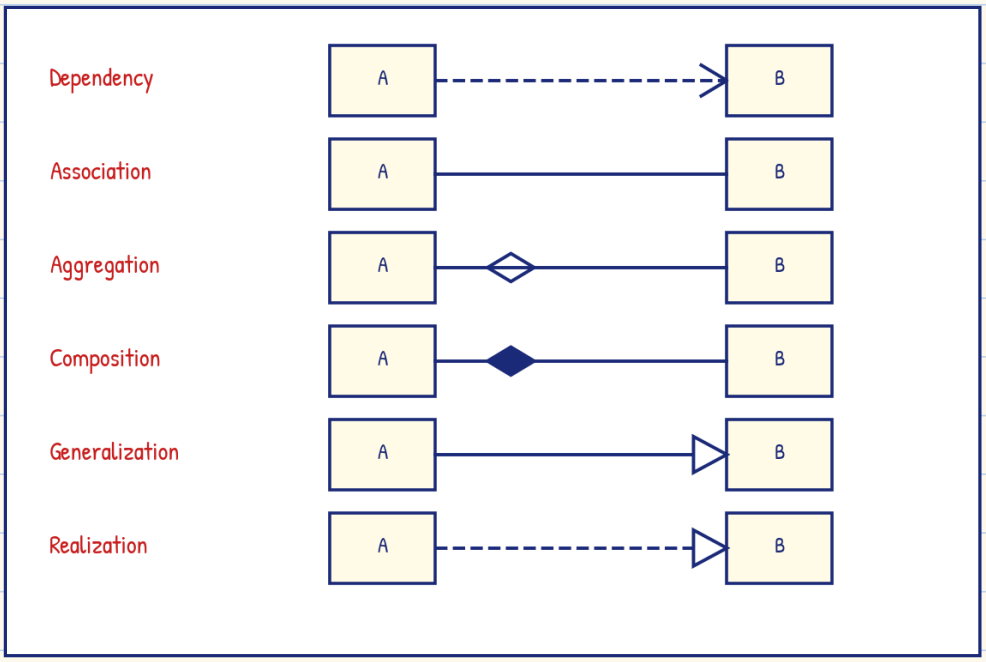
- Boxes into which a model can be decomposed
- Package - a general-purpose mechanism for organizing elements into groups; structural and behavioral things can be placed in a package; a package is purely conceptual; drawn as a tabbed folder with its name

Annotational Things

- Comments applied to describe, illuminate and remark about any element in a model
- Note - a symbol for rendering constraints and comments attached to an element or a collection of elements; drawn as a rectangle with a dog-eared corner

Relationships in the UML

- A relationship is a connection among things
- Four kinds of relationships: Dependency, Association, Generalization, Realization
- Types of class relationships shown in diagrams: association, composition, aggregation, generalization, dependency, navigation



Dependency

- A semantic relationship between two things; always directed
- A change to one thing (the independent thing) may affect the semantics of the other thing (the dependent thing)
- Drawn as a dashed line, possibly directed, occasionally with a label

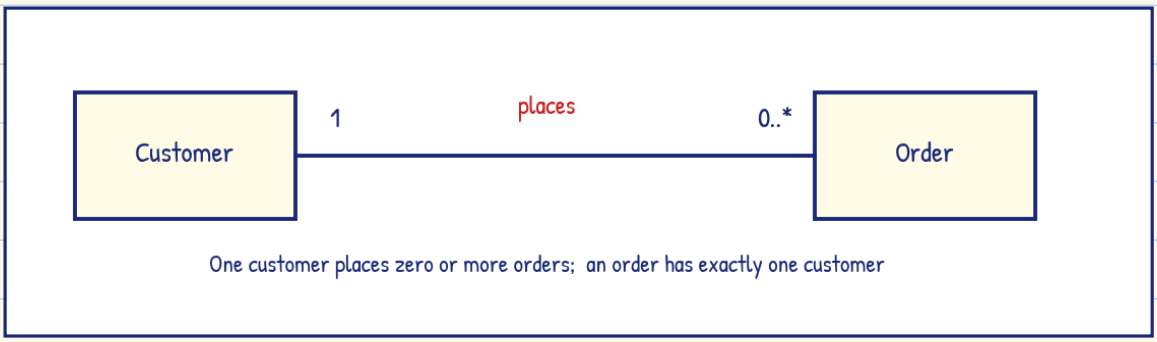
Association

- A structural relationship that describes a set of links; a link is a connection among objects
- Used when you want to show structural relationships; denotes a permanent, structural relationship
- Drawn as a solid line, possibly directed, occasionally with a label and often with other adornments such as multiplicity and role names
- Association names and roles can be written on the line

Multiplicity

- The multiplicity of a property is an indication of how many objects may fill the property
- Most common multiplicities:
  - 1 - an order must have exactly one customer
  - 0..1 - a corporate customer may or may not have a single sales rep
  - * - a customer need not place an order: zero or more orders
- Multiplicity is defined with a lower bound and an upper bound; the lower bound may be any positive number or zero; the upper is any positive number or * (unlimited)
- * is short for 0..*, and 1 is equivalent to 1..1
- Optional - implies a lower bound of 0; Mandatory - lower bound of 1 or more

- Single-valued - upper bound of 1; Multivalued - upper bound of more than 1, usually *



Bidirectional and Binary Associations

- Bidirectional association - a pair of properties that are linked together as inverses
- Binary association - an association that connects exactly two classes

Aggregation

- A special kind of association representing a structural whole-part relationship
- Represents a 'has-a' relationship: objects of the whole have objects of the part
- Drawn as a plain association with an open diamond at the whole end

Generalization

- Objects of the specialized element (the child) are substitutable for objects of the generalized element (the parent)
- The child shares the structure and behavior of the parent; a class may have zero, one or more parents
- Drawn as a solid line with a hollow arrowhead pointing to the parent

Realization

- A semantic relationship between classifiers in which one classifier specifies a contract that another classifier guarantees to carry out
- Occurs in two places: between interfaces and the classes or components that realize them; between use cases and the collaborations that realize them
- Drawn as a cross between a generalization and a dependency relationship (dashed line with a hollow arrowhead)

Diagrams in the UML

- A diagram is the graphical presentation of a set of elements, most often a connected graph of vertices (things) and arcs (relationships)
- The UML has nine diagrams: Class, Object, Use Case, Sequence, Collaboration, Statechart, Activity, Component, Deployment
- UML provides both structural and behavioral diagrams

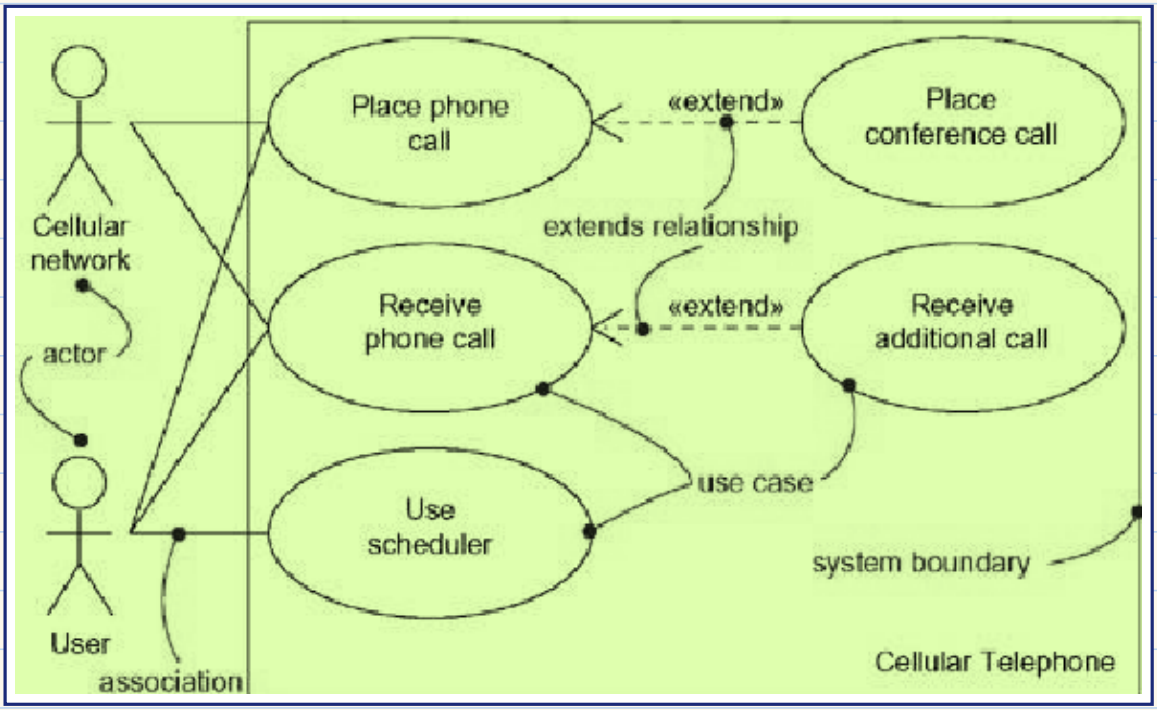
Lecture 7: Use Case Diagram

Behavioral Diagrams

- Diagrams used to visualize, specify, construct and document the dynamic aspects of the system (the dynamic aspects of a house include the airflow through its rooms)
- The UML has five behavioral diagrams: Use Case, Sequence, Collaboration, Statechart, Activity

Use Case Diagram

- Lets you view the whole system as a black box: you see what is outside the system and how the system reacts to those things, but not how it works inside
- A use case diagram commonly contains: use cases, actors, and dependency, generalization and association relationships



Common Uses of Use Case Diagrams

- To model the context of a system (from the actors):
  - Emphasizes the actors surrounding the system – draw a line around the whole system and state which actors lie outside and interact with it
  - Shows which actors carry out which use cases
- To model the requirements of a system (from the events):
  - Specify what the system should do from an outside point of view, independent of how it does it
  - Specify the desired behavior of the system
  - Shows which use cases include other use cases

Elements of a Use Case Diagram

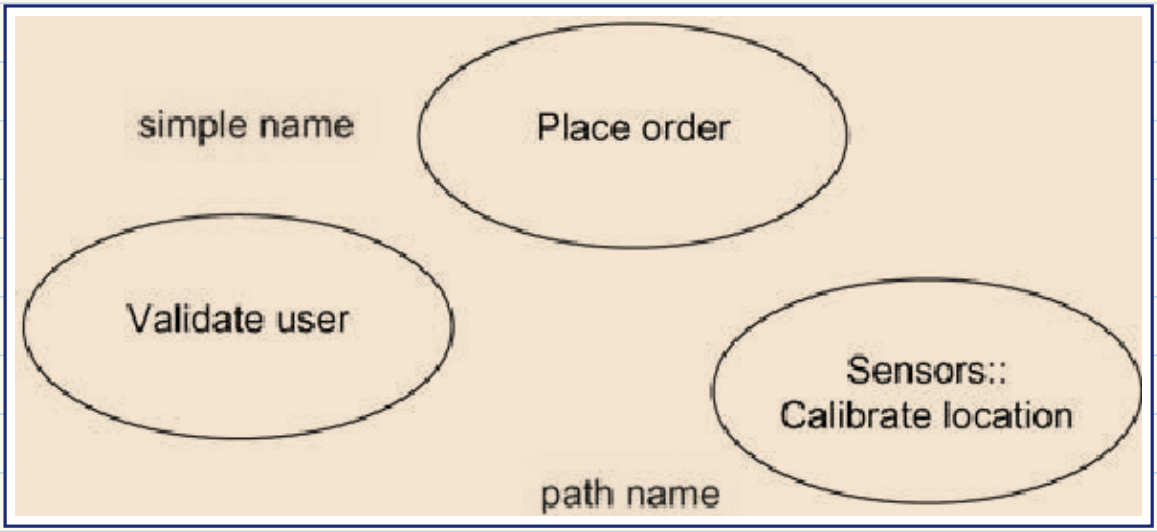
- Use case – a sequence of actions, including variants, that the system can perform together with

the actors

- Actor - a coherent set of roles played by a user when interacting with the system
- System boundary - the limit between the physical system and the actors that interact with it

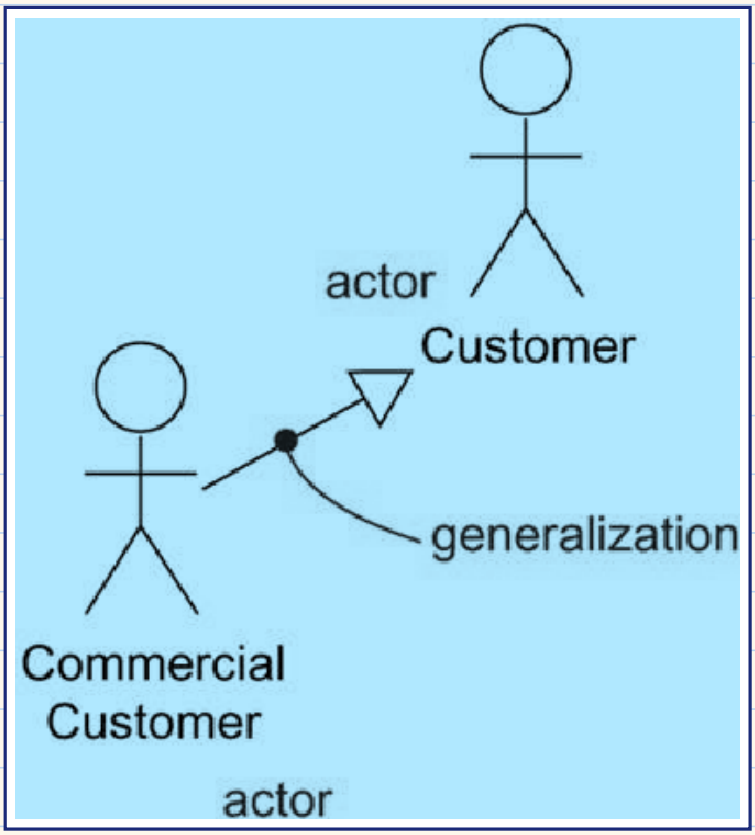
Use Case

- A technique for capturing the functional requirements of a system; represents an external view of the system
- A use case does not describe user interfaces, performance goals or other non-functional requirements



Actors

- An actor represents a coherent set of roles that users of use cases play when interacting with them
- An actor is anyone or anything that interacts with the system, causing it to respond to events
- Typically an actor is a role played by a human, a hardware device or even another system
- Actors live outside the system; actors can be related using generalizations; drawn as stick figures



Identifying Participants

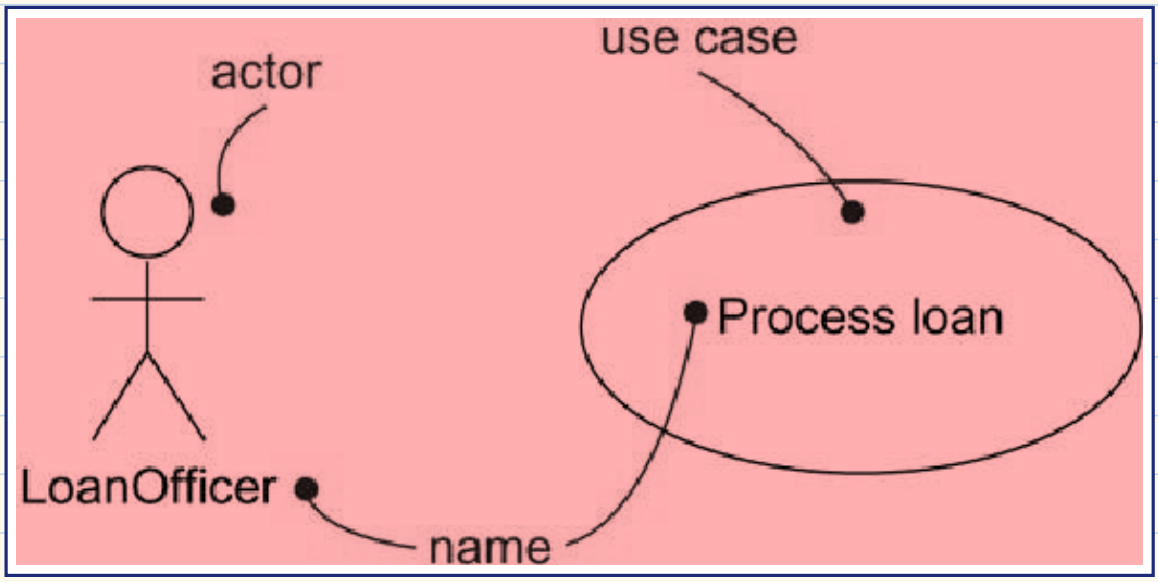
- Ask these questions:
  - What users or groups use the system to perform a task?
  - What users or groups are needed so the system can perform its functions?
  - What external systems use the system to perform a task?
  - What external systems, users or groups send information to the system?
  - What external systems, users or groups receive information from the system?

Initiator, Participant, Primary, Secondary

- Initiator vs participant: when a use case has more than one actor, the one that generates the stimulus is the initiator and the others are participants
- Primary vs secondary: the actor that directly interacts with the system is the primary actor; the others are secondary actors

Use Cases and Actors

- Actors may be connected to use cases only by association
- An association between an actor and a use case means they communicate with one another, each possibly sending and receiving messages

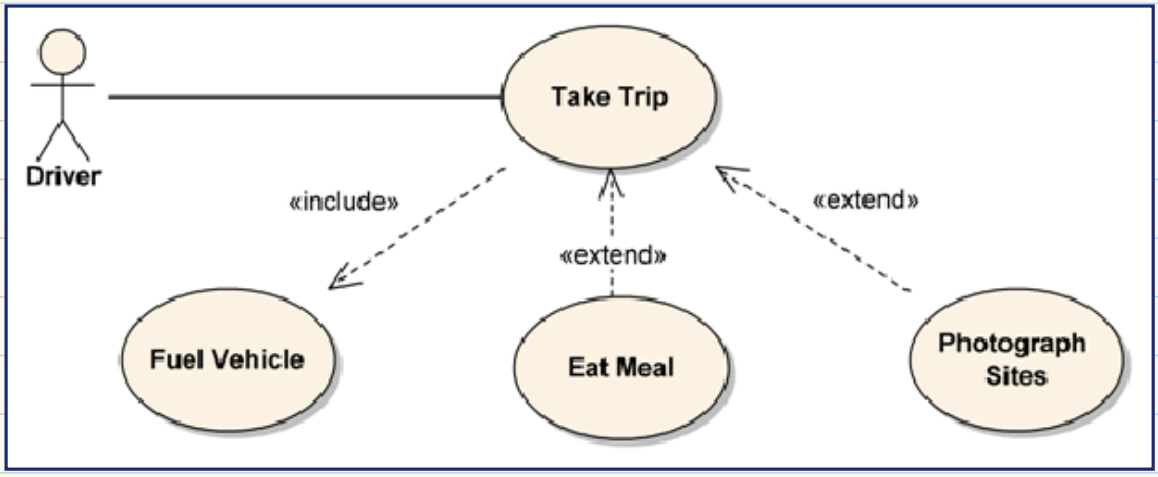


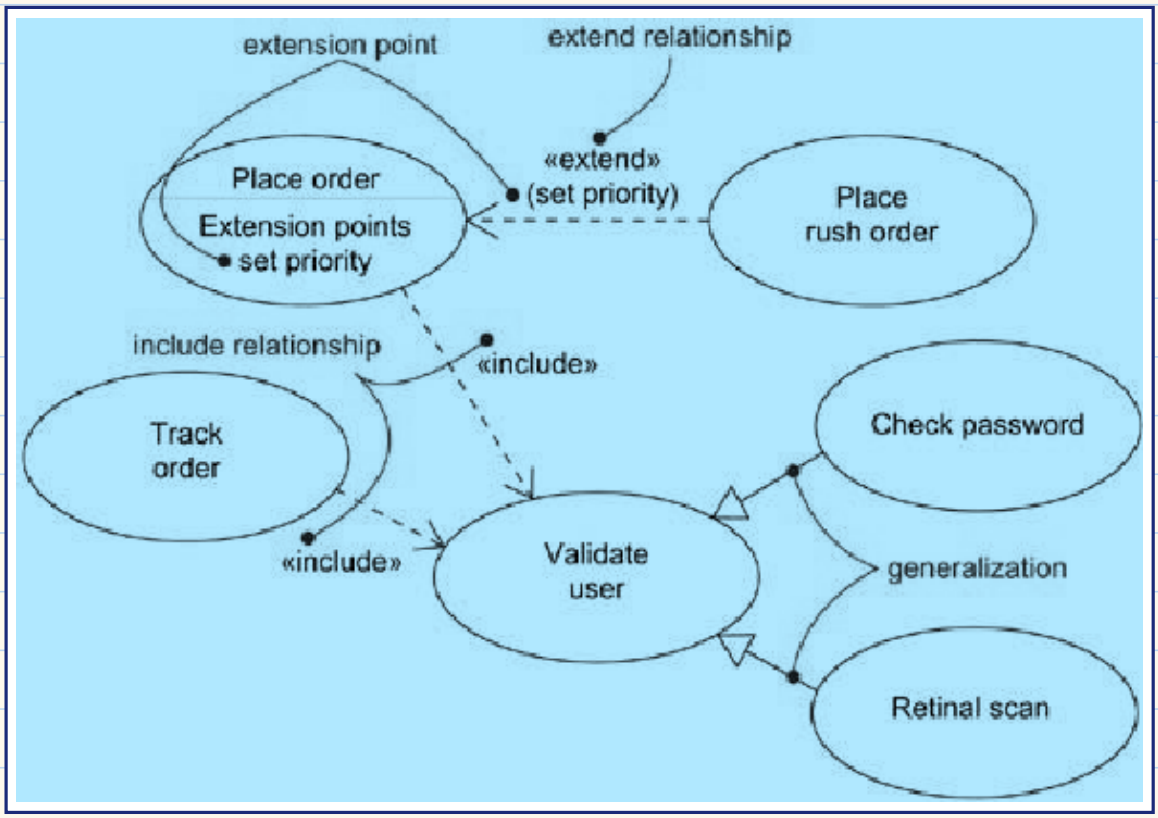
High Level Use Case Examples

- Buy items – actors: Customer, Cashier. A customer arrives at a checkout with items; the cashier records them and collects payment; the customer leaves with the items
- View Profile – actors: Doctor, Patient. The doctor updates his profile information; the patient reads the doctor's details

Include and Extend Stereotypes

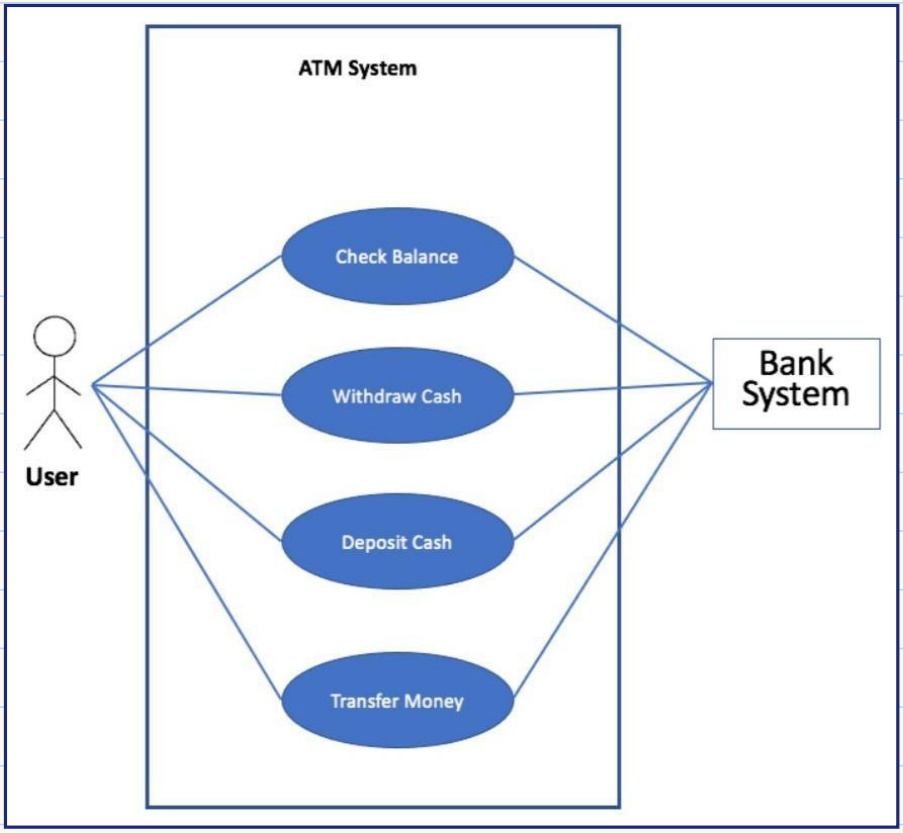
- Include – the behavior of a use case includes that of a base use case; the included use case never stands alone and is only instantiated as part of the larger base that includes it
  - Example: Place Order <<include>> Validate Payment – validation always happens when an order is placed
- Extend – the behavior of a use case extends that of a base use case; the base use case may stand alone, and under certain conditions its behavior is extended by another use case
  - Example: Reset Password <<extend>> Login – used only when, for example, the password is entered wrongly many times

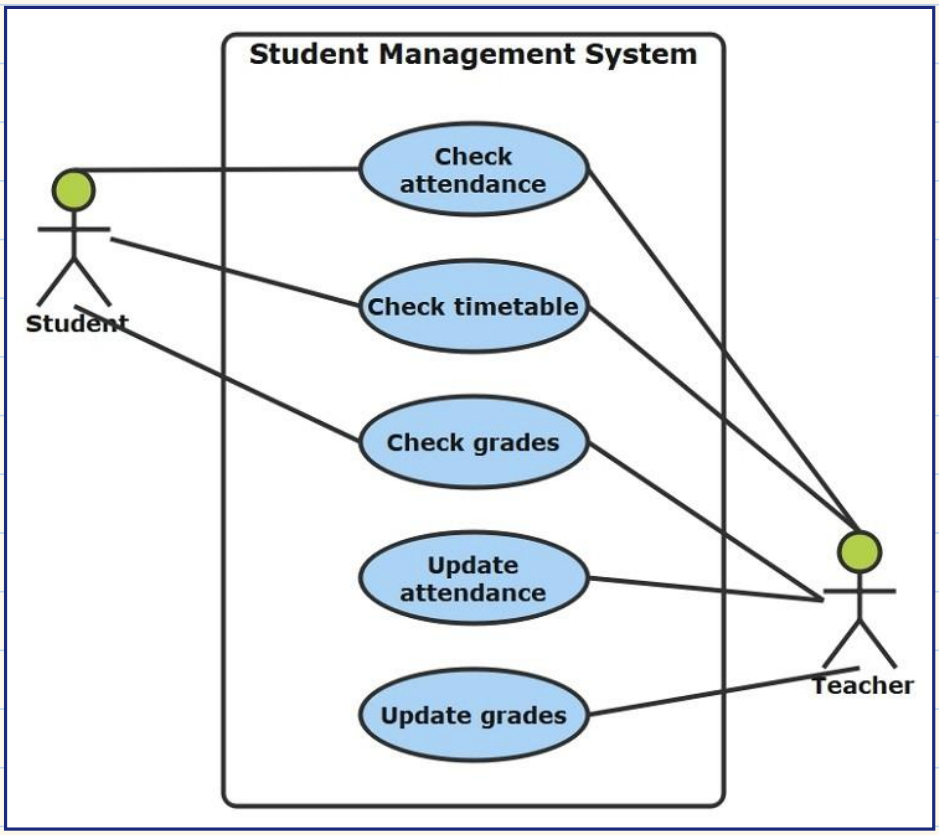


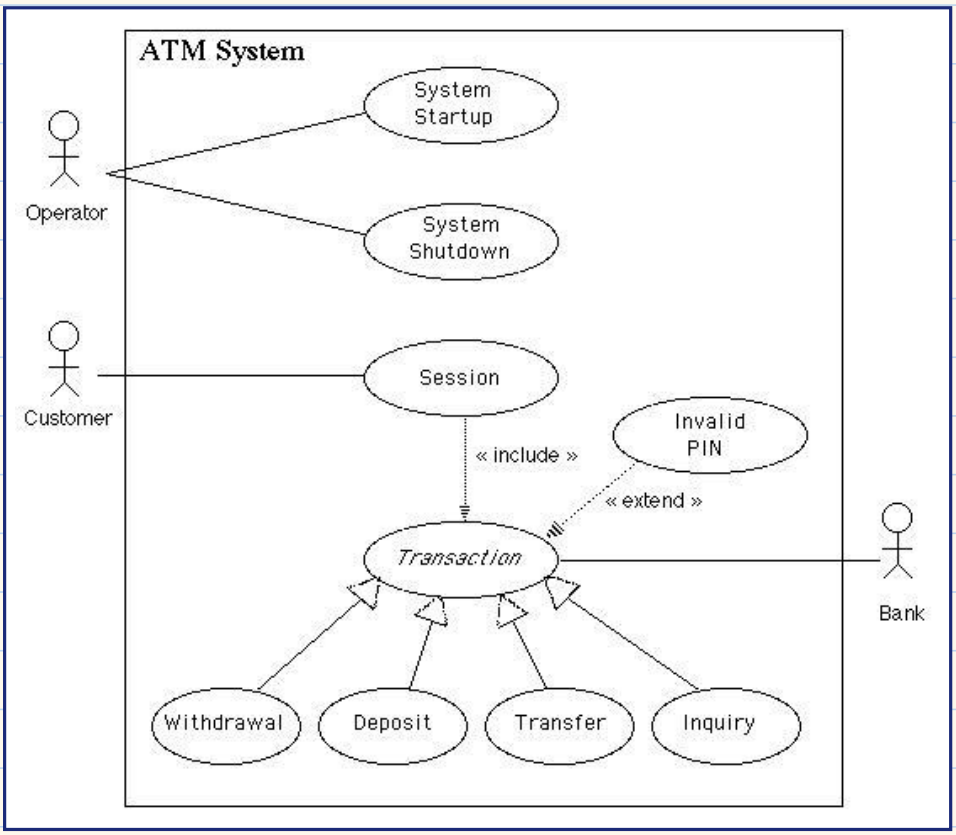


Examples of Use Case Diagram

- Examples of use case diagrams given in the lecture







Summary: Use Case Building Blocks

- Elements:
  - Actor – any external entity: hardware like a scanner, servers and clients, human users

- Use case - functionality
- Relationships:
  - Dependency between use cases: includes, extends
  - Association between actor and use case
  - Generalization between actors, and between use cases

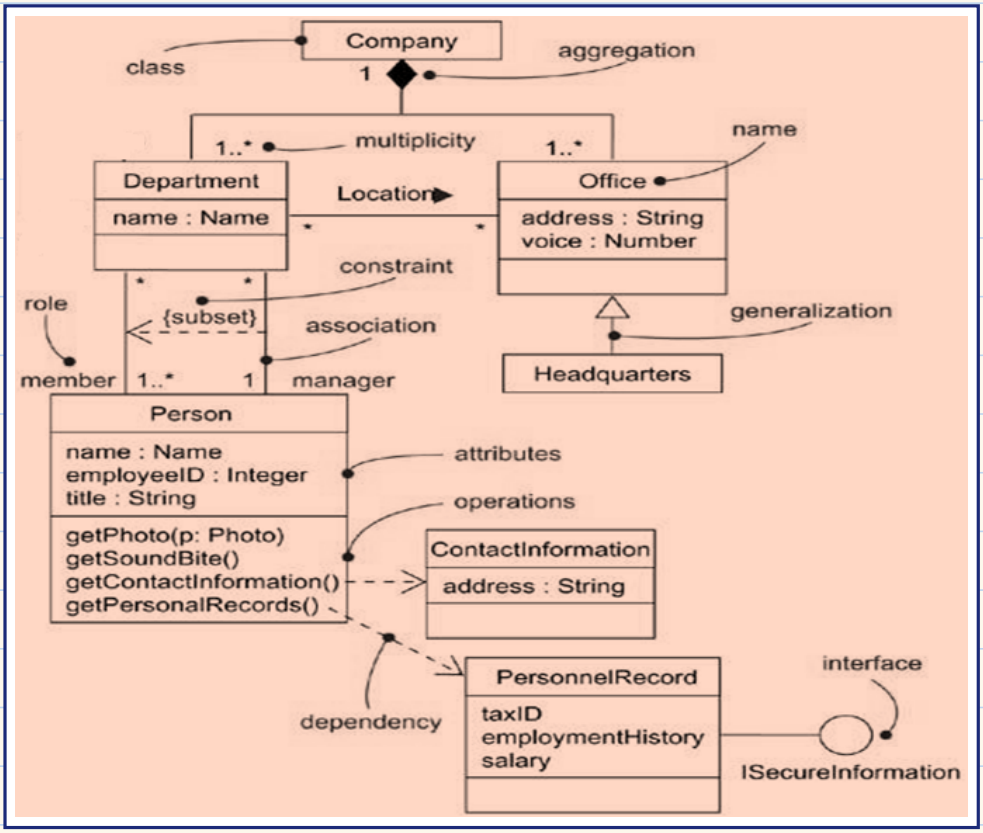
Lecture 8: Class Diagrams

Structural Diagrams

- Diagrams used to visualize, specify, construct and document the static aspects of the system (the static aspects of a house are its walls, doors, windows, pipes, etc.)
- The UML has four structural diagrams: Class, Object, Component, Deployment

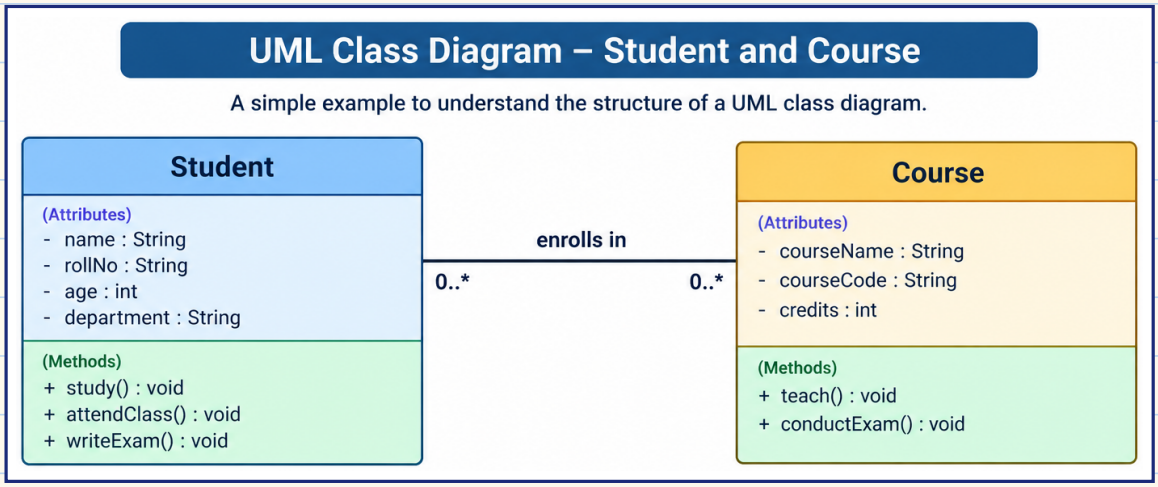
Class Diagram

- Addresses the static design view of a system; graphically it is a collection of vertices and arcs
- Supports the functional requirements of the system; most UML diagrams are class diagrams
- It describes the types of objects in the system and the various static relationships among them
- It shows the properties and operations of a class and the constraints that apply to the way objects are connected
- A class diagram is made up of classes, interfaces and packages



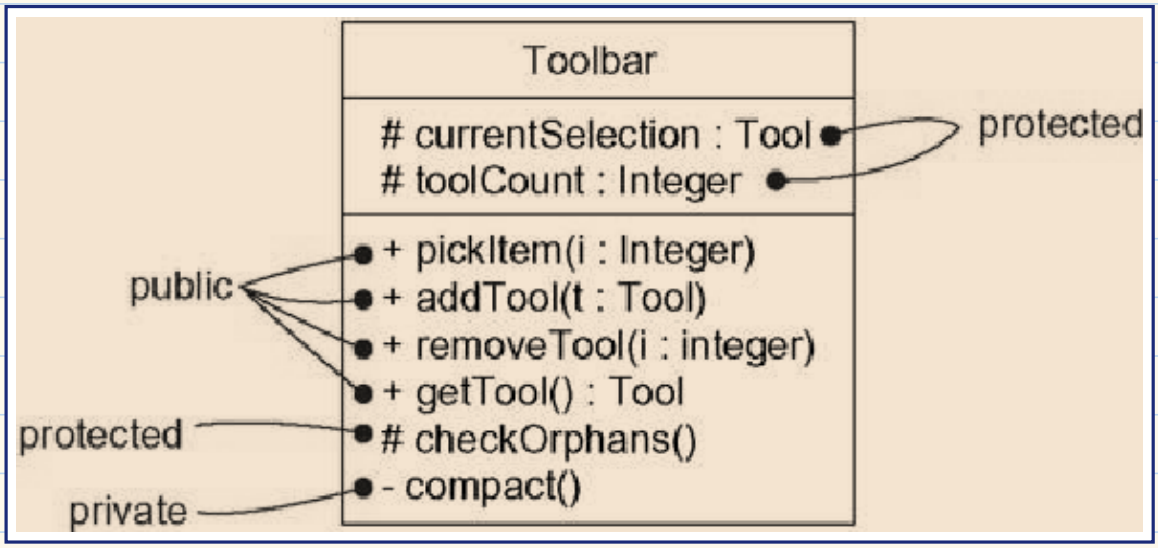
Example of Class Diagram

- Example given in the lecture: Student and Course classes with attributes, methods and the 'enrols in' association (0..* on both sides)



Visibility

- The visibility of a feature specifies whether it can be used by other classifiers (classes, interfaces, data types, signals, components, nodes, use cases, subsystems)
- Three levels of visibility:
  - Public (+) - any outside classifier with visibility to the given classifier can use the feature
  - Protected (#) - any descendant of the classifier can use the feature
  - Private (-) - only the classifier itself can use the feature



Relationships between Classes

- Association - relationship between peers
- Generalization - the reverse of inheritance (child to parent)
- Dependency - usage of one class's object in another class's functions

- Realization - classes realize interfaces
- Composition - a set of objects make up a class
- Aggregation - a collection of things that are not mandatory